

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Lab Manual 08

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives.....	4
1 Handling Missing Values.....	4
1.1 Detecting Missing Values.....	4
1.2 Handling Missing Values.....	5
2 Removing Duplicate Data.....	5
3 Data Type Conversion.....	5
3.1 Fixing Data Types.....	5
4 Feature Scaling.....	7
4.1 Normalization.....	7
4.2 Standardization.....	8
4.3 Feature Scaling During Testing Time.....	8
4.4 When to Use Normalization vs Standardization.....	9
5 Encoding.....	10
5.1 Label Encoding.....	10
5.2 One-Hot Encoding.....	11
5.3 Ordinal Encoding.....	12
5.4 Encoding at Testing Time.....	13
6 What is Machine learning?.....	14
6.1 Types of Machine Learning.....	14
6.2 Supervised Learning.....	15
7 What is Scikit-learn?.....	16
7.1 Basic Workflow in Scikit-learn.....	19
8 Classification.....	21
8.1 Import Libraries.....	21
8.2 Load the Dataset.....	22
8.3 Split Data (Train/Test).....	22
8.4 Feature Scaling.....	22
8.5 Train the Classifier.....	22
8.6 Make Predictions.....	23
8.7 Evaluate the Model.....	23
8.8 Trying Another Classifier.....	24
9 Regression.....	24
9.1 Import Libraries.....	24
9.2 Load the Dataset.....	24
9.3 Split Data (Train/Test).....	25
9.4 Feature Scaling.....	25
9.5 Train the Regression Model.....	25
9.6 Make Predictions.....	25
9.7 Evaluate the Model.....	26

9.8 Trying Another Regression Model.....	26
10 Evaluation metrics for Regression.....	26
10.1 Mean Absolute Error (MAE).....	27
10.2 Mean Squared Error (MSE).....	27
10.3 Root Mean Squared Error (RMSE).....	28
10.4 R-squared (R^2) Score.....	28
11 Evaluation metrics for Classification.....	29
11.1 Confusion Matrix.....	29
11.2 Accuracy.....	30
11.3 Precision.....	31
11.4 Recall.....	31
11.5 F1 Score.....	32
11.6 Area Under Curve (AUC) and ROC Curve.....	32
11.7 Classification Report.....	33

Objectives

After performing this lab, students shall be able to understand the following data science concepts and applications:

- ✓ Data preprocessing
- ✓ Supervised learning
- ✓ Model Evaluation

1 Handling Missing Values

Missing values occur when no data is recorded for certain entries in a dataset. This can happen due to data collection errors, incomplete surveys, or system failures.

City	State	Zip
Chicago	IL	<i>null</i>
San Antonio	TX	22259
<i>null</i>	TX	77343
<i>null</i>	TX	04927
San Diego	CA	16150
Philadelphia	PA	<i>null</i>
New York	<i>null</i>	25235
San Antonio	<i>null</i>	73868
Los Angeles	CA	24607
San Diego	CA	39289

Missing data can introduce bias, lead to incorrect conclusions, and cause errors in computations.

1.1 Detecting Missing Values

```
import pandas as pd

df = pd.read_csv("data.csv") # Load dataset
print(df.isnull().sum())    # Count missing values in each column
print(df.info())            # Check data types and missing values
```

1.2 Handling Missing Values

1.2.1 Remove rows/columns

Remove rows/columns with too many missing values:

```
df.dropna(inplace=True) # Remove rows with missing values
df.dropna(axis=1, inplace=True) # Remove columns with missing values
```

1.2.2 Imputation

Impute missing values (Mean/Median/Mode)

```
df['Age'].fillna(df['Age'].median(), inplace=True) # Fill missing
values with median
df['City'].fillna(df['City'].mode()[0], inplace=True) # Fill missing
values with mode
```

2 Removing Duplicate Data

Duplicates can inflate dataset size and lead to incorrect insights.

```
df.duplicated().sum() # Check for duplicate rows
df = df.drop_duplicates() # Remove duplicate rows
```

3 Data Type Conversion

Data type conversion is the process of changing a data column's type (like from text to number or string to date). In data analysis or machine learning, having the correct data types is crucial. For example:

- If prices are stored as strings ("123.45"), you can't calculate totals or averages correctly.
- If dates are stored as text ("2023-01-01"), you can't sort them or calculate durations properly.

3.1 Fixing Data Types

3.1.1 pd.to_numeric() → Convert to numeric (int or float)

```
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
```

Convert strings like "123.45" or "1,000" to numeric values.

Options for errors:

Value	Description
'raise' (default)	Raise an error if conversion fails.
'coerce'	Invalid parsing will be set as NaN.
'ignore'	Return original data if parsing fails.

3.1.2 `pd.to_datetime()` → Convert to datetime

```
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

Supports various date formats automatically.

3.1.3 `.astype()` → Convert to specific type

```
df['Quantity'] = df['Quantity'].astype(int)
```

Explicitly cast a column to a specific type (int, float, str, etc.)

Can be used for conversions like:

- int → integer
- float → floating point
- str → string
- category → efficient for categorical data

Will raise an error if conversion fails. So make sure the data is clean or use `errors='coerce'` first.

3.1.4 `infer_objects()` → Let Pandas guess better data types

```
df = df.infer_objects()
```

If your DataFrame has object types, this tries to convert them to more specific types like int, float, or datetime based on content.

3.1.5 `convert_dtypes()` → Automatically assign best data types

```
df = df.convert_dtypes()
```

Automatically converts all columns to the best possible data type (including new nullable types like Int64, string, boolean, etc.)

Especially useful when loading raw data from CSV/Excel files.

4 Feature Scaling

Feature scaling is a data preprocessing technique used to bring numerical features onto a similar scale. It is especially important for machine learning algorithms that rely on distance, gradients, or variance, such as k-NN, SVM, linear regression, and neural networks. Without feature scaling, variables with larger ranges can dominate the learning process and lead to poor model performance.

Feature scaling:

- Improves model convergence speed
- Prevents bias toward large-magnitude features
- Helps distance-based algorithms work correctly
- Stabilizes gradient descent optimization

Tree-based models (e.g., Decision Trees, Random Forests) usually do not require feature scaling.

4.1 Normalization

Normalization rescales data to a fixed range, typically between 0 and 1. It preserves the shape of the original distribution but compresses the scale.

Normalization is useful when:

1. Data does not follow a normal distribution (skewness...)
2. You are using distance-based algorithms (k-NN, k-means)
3. Feature bounds are known and meaningful

It is calculated using:

$$X_{\text{new}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Sample data
X = np.array([[10], [20], [30], [40], [50]])

# Normalization
scaler = MinMaxScaler()
```

```
X_normalized = scaler.fit_transform(X)
print(X_normalized)
```

4.2 Standardization

Standardization transforms data to have:

- Mean = 0
- Standard deviation = 1

It does not restrict values to a fixed range and works well when data follows a normal distribution.

It is calculated using:

$$X_{\text{new}} = (X - \text{mean}) / \text{Std}$$

Standardization is preferred when:

- Data contains outliers
- Features follow a Gaussian distribution
- Using algorithms like linear regression, logistic regression, SVM, or PCA

```
from sklearn.preprocessing import StandardScaler

# Sample data
X = np.array([[10], [20], [30], [40], [50]])

# Standardization
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
print(X_standardized)
```

4.3 Feature Scaling During Testing Time

During testing (or prediction) time, feature scaling does not change its formula, but how it is applied changes.

- The scaler is fit only on the training data
- During testing, the scaler is reused, not refit
- The same parameters learned from training are applied to test data

This ensures fairness and prevents data leakage.

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# Sample dataset
X = np.array([[10], [20], [30], [40], [50]])

# Train-test split
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

# Fit scaler on training data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Apply same scaler to test data
X_test_scaled = scaler.transform(X_test)

print("Train scaled:\n", X_train_scaled)
print("Test scaled:\n", X_test_scaled)

```

4.4 When to Use Normalization vs Standardization

The choice between normalization and standardization depends mainly on the distribution of the data and the presence of skewness. If a feature is approximately normally distributed (symmetric and bell-shaped), standardization is preferred because it centers the data around zero and scales it based on variance. If the data is not normally distributed or has clear bounds, normalization is often more suitable, especially for distance-based algorithms.

Skewness is a key indicator in this decision. Skewness measures the asymmetry of a distribution and can be visually checked using a histogram or numerically computed using the `.skew()` function. A skewness value close to 0 indicates a symmetric distribution. In practice, if skewness lies roughly between -0.5 and $+0.5$, the data can be treated as approximately normal and standardization is recommended. If skewness is greater than $+1$ or less than -1 , the distribution is highly skewed, and normalization is generally preferred unless the data is transformed first (e.g., log transformation).

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import skew

```

```
# Sample data
data = np.array([10, 12, 15, 18, 20, 22, 25, 100])

# Histogram
plt.hist(data, bins=10)
plt.title("Histogram of Feature")
plt.show()

# Skewness value
skew_value = skew(data)
print("Skewness:", skew_value)
```

Rule of thumb:

- Skewness between -0.5 and $+0.5$: safe to use standardization
- Skewness > 1 or < -1 : prefer normalization

5 Encoding

Encoding is a feature engineering technique used to convert categorical variables into numerical form. Since most machine learning algorithms operate only on numerical data, categorical features such as colors, countries, or severity levels must be encoded before model training.

The choice of encoding method depends on whether the categorical data has no order (nominal) or a meaningful order (ordinal).

5.1 Label Encoding

Label Encoding converts each category into a unique integer value. The entire categorical column is replaced by these numeric labels.

For example:

- Red: 0
- Green: 1
- Blue: 2

Label encoding is suitable when:

- The data is ordinal (has a natural order)
- You are using tree-based models (Decision Trees, Random Forests)
- The numeric ordering will not mislead the model

Using label encoding on nominal data can be dangerous, as models may incorrectly assume a mathematical relationship between categories.



```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Categorical data
colors = ['Red', 'Blue', 'Green', 'Blue', 'Red']
df = pd.DataFrame({'Color': colors})

label_encoder = LabelEncoder()
df['Color_encoded'] = label_encoder.fit_transform(df['Color'])
print(df)
```

5.2 One-Hot Encoding

One-Hot Encoding converts a categorical feature into multiple binary columns, one for each unique category. A value of 1 indicates the presence of that category, while 0 indicates absence.

For example, a Country feature with values USA, Canada, and India becomes three separate columns.

One-hot encoding is best when:

- The data is nominal (no inherent order)
- The number of unique categories is small
- You are using models like logistic regression, KNN, or neural networks

Its main drawback is increased dimensionality, which can lead to sparse data.



Color			
Red			
Red			
Yellow			
Green			
Yellow			

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

```
import pandas as pd

countries = ['USA', 'Canada', 'India', 'USA', 'Canada']
df = pd.DataFrame({'Country': countries})

# One-Hot Encoding
one_hot = pd.get_dummies(df['Country'])
print(one_hot)
```

5.3 Ordinal Encoding

Ordinal Encoding assigns numerical values to categories based on a predefined order. Unlike label encoding, the order is explicitly defined by the user, making it safer and more meaningful for ordinal data.

For example:

- Low: 1
- Medium: 2
- High: 3

Ordinal encoding should be used when:

- Categories have a clear and meaningful ranking
- The order itself carries information
- You want to preserve relative importance between categories

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Ordinal data
severity = ['Low', 'Medium', 'High', 'Medium', 'Low']
df = pd.DataFrame({'Severity': severity})

encoder = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
df['Severity_encoded'] = encoder.fit_transform(df[['Severity']])
print(df)
```

5.4 Encoding at Testing Time

During testing or prediction time, encoding methods must not be refit. Just like feature scaling, encoders are learned from the training data and reused unchanged on test data. This ensures consistency and prevents data leakage.

The key rule is:

fit on training data, transform on test data.

So instead of using `.fit_transform`, you only need to use `.transform`. Consider the example below:

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# Training data
train = pd.Series(['Low', 'Medium', 'High'])

# Testing data
test = pd.Series(['Medium', 'Low'])
```

```

encoder = LabelEncoder()

# Fit on training data
encoder.fit_transform(train)

# Transform test data
test_encoded = encoder.transform(test)
print(test_encoded)

```

6 What is Machine learning?

Machine Learning (ML) is a field of computer science that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for each task. Instead of manually writing rules, we allow the system to learn those rules automatically from examples.

Traditional Programming

- Input data + Fixed rules = Output
- Rules are written by humans

Machine Learning

- Input data + Correct outputs = Model learns rules
- Rules are learned automatically from data

Some real world examples of machine learning include:

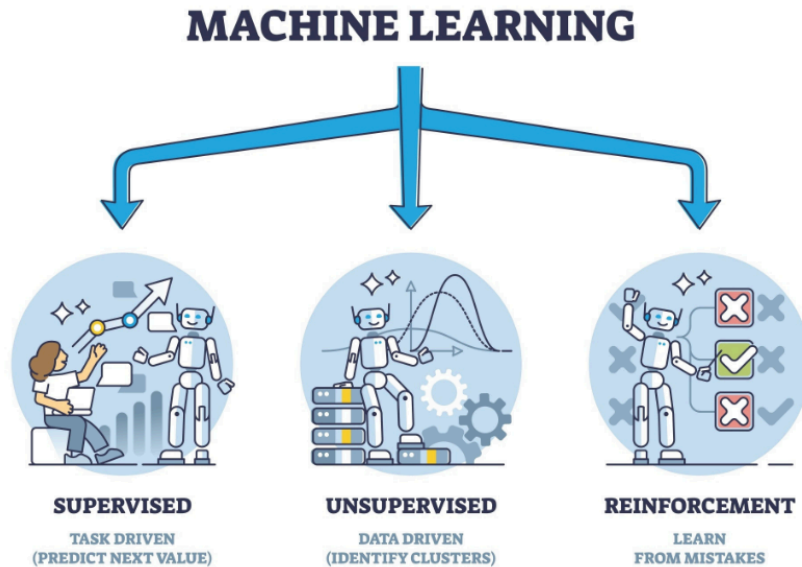
- Spam email detection
- Medical diagnosis
- House price prediction
- Credit card fraud detection
- Recommendation systems

6.1 Types of Machine Learning

Machine learning is generally divided into three categories:

Type	Description	Example
Supervised Learning	Data includes correct answers	Predict exam score

Unsupervised Learning	No labeled output	Customer grouping
Reinforcement Learning	Learning via rewards	Game playing AI



6.2 Supervised Learning

Supervised learning means that we have input variables (features): X and an output variable (target/label): y . The model learns a function:

$$f(X) = y$$

Key Terms

- Feature: Input variable (e.g., house size)
- Target: Output variable (e.g., house price)
- Dataset: Collection of samples
- Model: Algorithm that learns patterns
- Training: Learning from known data
- Testing: Evaluating on unseen data

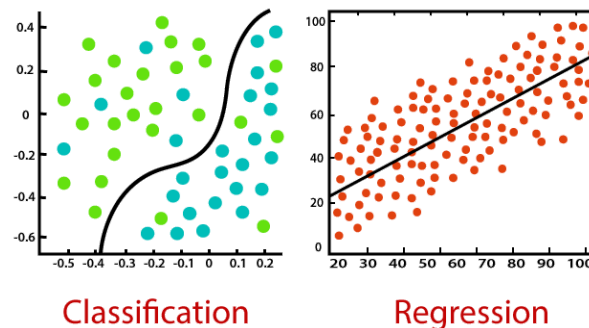
Supervised learning has two main categories. Each of them is explained below.

6.2.1 Regression

Regression is a type of supervised learning used when the target variable is continuous. A continuous variable can take any numerical value within a range. For example, predicting house prices, estimating temperatures, forecasting sales revenue, or estimating a student's final exam score are all regression problems.

6.2.2 Classification

Classification is a type of supervised learning used when the target variable is categorical. In classification problems, the goal is to assign an input example to one of several predefined classes. For example, determining whether an email is spam or not spam, predicting whether a patient has a disease or not, or classifying images of animals into categories are all classification tasks.



7 What is Scikit-learn?

Scikit-learn (also written as sklearn) is an open-source machine learning library for Python. It provides simple and efficient tools for building, training, and evaluating machine learning models.

It is built on top of:

- NumPy (numerical computation)
- Pandas (data handling)
- Matplotlib (visualization)

Before libraries like Scikit-learn, implementing machine learning algorithms required writing complex mathematical code from scratch.

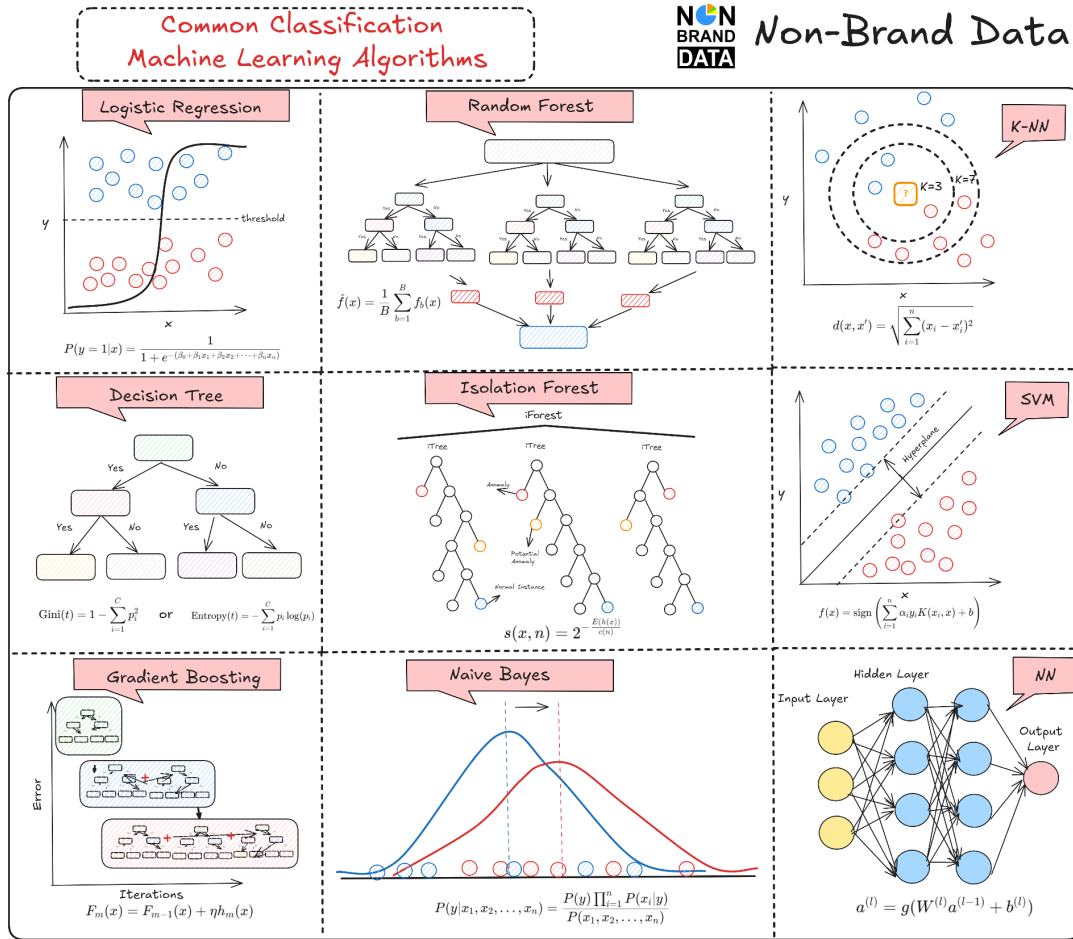
With Scikit-learn, we can:

- Load datasets easily
- Preprocess data
- Train models in a few lines of code

- Evaluate performance
- Compare different algorithms

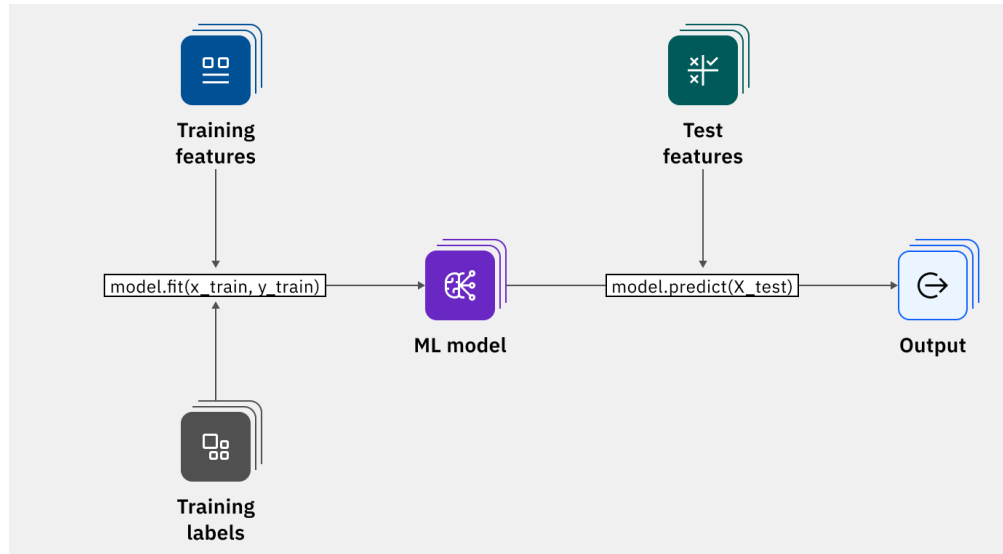
Scikit-learn supports many machine learning tasks such as:

- Supervised Learning
 - Linear Regression
 - Logistic Regression
 - K-Nearest Neighbors
 - Decision Trees
 - Random Forest
 - Support Vector Machines
- Unsupervised Learning
 - K-Means Clustering
 - Hierarchical Clustering
 - PCA (Dimensionality Reduction)
- Preprocessing Tools
 - Scaling features
 - Encoding categorical variables
 - Handling missing values
- Model Evaluation
 - Accuracy
 - Precision, Recall
 - Confusion Matrix
 - Cross-validation

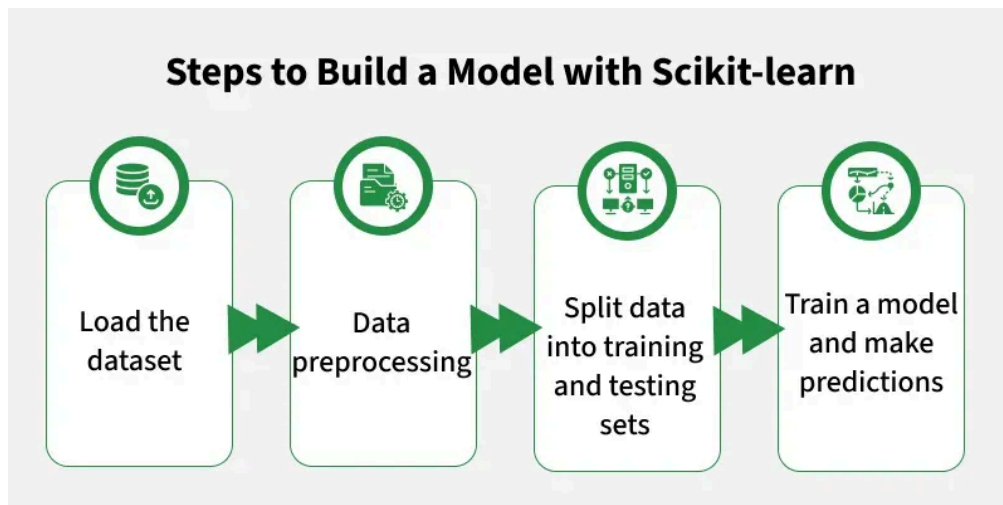


All models in Scikit-learn follow the same structure

Method	Purpose
fit()	Train the model
predict()	Make predictions
score()	Evaluate performance



7.1 Basic Workflow in Scikit-learn



We have already covered the first 2 steps in previous labs.

7.1.1 Split Data (Train/Test)

In machine learning, our goal is not just to build a model that performs well on known data, but one that performs well on new, unseen data.

If we train and test the model using the same dataset, the model may memorize the data and the performance may look very high but it may fail in real-world scenarios.

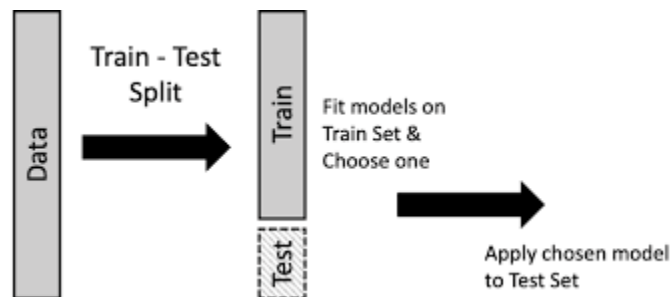
To avoid this, we divide the dataset into two parts:

- **Training Set:** The training set is used to:
 - Teach the model
 - Learn patterns
 - Estimate parameters (weights)

During training, the model:

- Looks at input features (X)
- Compares predictions with actual outputs (y)
- Adjusts its parameters to reduce error
- **Testing Set:** The testing set is used to:
 - Evaluate model performance
 - Measure generalization ability
 - Simulate real-world prediction

The model must never learn from the test data. The test data must remain unseen during training.



Suppose we have 100 student records. If we split 80–20:

- 80 records: Training set
- 20 records: Testing set

The model learns from the 80 students and is evaluated on the 20 new students.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Parameter	Meaning
X	Features
y	Target
test_size=0.2	20% data used for testing
random_state=42	Ensures reproducibility

When splitting data:

- The split is random.
- Different runs may give different results.

By setting random_state=42, we ensure:

- Same split every time
- Reproducible results
- Fair comparison between models

7.1.2 Model Training

We will be covering 2 types of models: classification and regression.

8 Classification

In this section, we will go through an example of how we can train and test a classification model step by step.

8.1 Import Libraries

```
import numpy as np
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
```

```
classification_report
```

8.2 Load the Dataset

```
iris = load_iris()

X = iris.data      # Features
y = iris.target    # Labels

print(X.shape)    # (150 samples, 4 features)
print(y.shape)    # (150,)
```

8.3 Split Data (Train/Test)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

8.4 Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

8.5 Train the Classifier

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression()
```

We are using logistic regression as a classifier here. You can explore other classifiers here: https://scikit-learn.org/stable/supervised_learning.html

What happens internally:

- The model learns decision boundaries
- It finds weights that best separate classes

8.6 Make Predictions

```
y_pred = model.predict(X_test)
```

8.7 Evaluate the Model

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 1.0

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

8.8 Trying Another Classifier

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

KNN Accuracy: 1.0

9 Regression

In this section, we will go through an example of how we can train and test a regression model step by step.

9.1 Import Libraries

```
import numpy as np
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

9.2 Load the Dataset

```
data = fetch_california_housing()

X = data.data      # Features
y = data.target    # Target

print(X.shape)     # (20640 samples, 8 features)
```

```
print(y.shape) # (20640,)
```

9.3 Split Data (Train/Test)

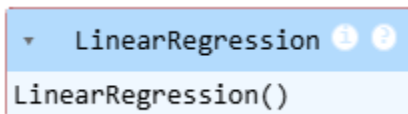
```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42  
)
```

9.4 Feature Scaling

```
scaler = StandardScaler()  
  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

9.5 Train the Regression Model

```
model = LinearRegression()  
model.fit(X_train, y_train)
```



We are using linear regression as a regressor here. You can explore other regressors here: https://scikit-learn.org/stable/supervised_learning.html

What happens internally:

- Model finds weights for each feature
- Minimizes the difference between predicted and actual values

9.6 Make Predictions

```
y_pred = model.predict(X_test)
```

9.7 Evaluate the Model

```
mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
```

MSE: 0.5558915986952442

```
rmse = np.sqrt(mse)
print("RMSE:", rmse)
```

RMSE: 0.7455813830127763

```
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

R² Score: 0.575787706032451

9.8 Trying Another Regression Model

```
from sklearn.tree import DecisionTreeRegressor

tree_model = DecisionTreeRegressor(random_state=42)
tree_model.fit(X_train, y_train)

y_pred_tree = tree_model.predict(X_test)

print("Decision Tree R2 Score:", r2_score(y_test, y_pred_tree))
```

Decision Tree R² Score: 0.6230424613065773

10 Evaluation metrics for Regression

Regression models predict continuous outcomes (e.g., house prices, temperatures). To measure

their performance, we need evaluation metrics that quantify how close predictions are to actual values. There is no single “best” metric; each has strengths and weaknesses.

10.1 Mean Absolute Error (MAE)

In the fields of statistics and machine learning, the Mean Absolute Error (MAE) is a frequently employed metric. It's a measurement of the typical absolute discrepancies between a dataset's actual values and projected values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |x_i - y_i|$$

Where:

- x_i represents the actual or observed values for the i -th data point.
- y_i represents the predicted value for the i -th data point.

```
from sklearn.metrics import mean_absolute_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

mae = mean_absolute_error(true_values, predicted_values)
print("Mean Absolute Error:", mae)
```

10.2 Mean Squared Error (MSE)

A popular metric in statistics and machine learning is the Mean Squared Error (MSE). It measures the square root of the average discrepancies between a dataset's actual values and projected values. MSE is frequently utilized in regression issues and is used to assess how well predictive models work.

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2$$

Where:

- x_i represents the actual or observed values for the i -th data point.
- y_i represents the predicted value for the i -th data point.

```
from sklearn.metrics import mean_squared_error
```

```

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

mse = mean_squared_error(true_values, predicted_values)
print("Mean Squared Error:", mse)

```

10.3 Root Mean Squared Error (RMSE)

RMSE stands for Root Mean Squared Error. It is a usually used metric in regression analysis and machine learning to measure the accuracy or goodness of fit of a predictive model, especially when the predictions are continuous numerical values.

$$RMSE = \sqrt{MSE}$$

```

from sklearn.metrics import mean_squared_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

rmse = np.sqrt(mean_squared_error(true_values, predicted_values))
print("Root mean Squared Error:", rmse)

```

10.4 R-squared (R²) Score

A statistical metric frequently used to assess the goodness of fit of a regression model is the R-squared (R²) score, also referred to as the coefficient of determination. It quantifies the percentage of the dependent variable's variation that the model's independent variables contribute to. R² is a useful statistic for evaluating the overall effectiveness and explanatory power of a regression model.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

```

from sklearn.metrics import r2_score

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

r2 = r2_score(true_values, predicted_values)
print("R-squared (R2) Score:", r2)

```

11 Evaluation metrics for Classification

We will be using the following sample dataset for the given example code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_auc_score,
    classification_report,
    roc_curve
)

# Example true labels and predicted labels
y_true = [0, 1, 0, 1, 0, 1, 1, 0]
y_pred = [0, 1, 0, 0, 0, 1, 1, 1]
y_pred_proba = [0.1, 0.9, 0.2, 0.4, 0.3, 0.8, 0.7, 0.6] # Probabilities
for ROC-AUC
```

11.1 Confusion Matrix

Confusion matrix creates a N X N matrix, where N is the number of classes or categories that are to be predicted.

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

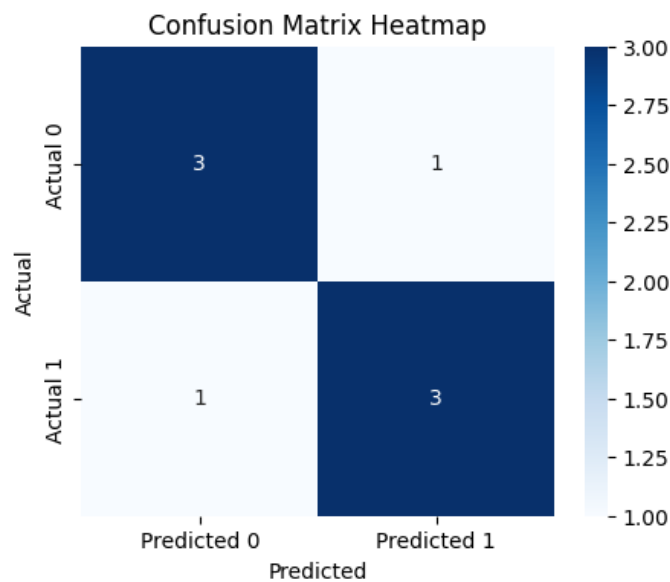
		Actual Value	
		Positive	Negative
Predicted Value	Positive	5600	600
	Negative	500	3300

- True Positives: Correctly predicted positive class.
- True Negatives: Correctly predicted negative class.
- False Positives: Incorrectly predicted positive class (Type I error)
- False Negatives: Incorrectly predicted negative class (Type II error)

```
cm = confusion_matrix(y_true, y_pred)
print("\nConfusion Matrix:\n", cm)
```

To visualize confusion matrix:

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix Heatmap')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```



11.2 Accuracy

Accuracy is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

While accuracy provides a quick snapshot, it can be misleading in cases of imbalanced datasets. For example, in a dataset with 90% class A and 10% class B, a model predicting only class A will still achieve 90% accuracy but it will fail to identify any class B instances.

Accuracy is good but it gives a False Positive sense of achieving high accuracy. The problem arises due to the possibility of misclassification of minor class samples being very high.

```
accuracy = accuracy_score(y_true, y_pred)
print("Accuracy:", accuracy)
```

11.3 Precision

It measures how many of the positive predictions made by the model are actually correct. It's useful when the cost of false positives is high such as in medical diagnoses where predicting a disease when it's not present can have serious consequences.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision helps ensure that when the model predicts a positive outcome, it's likely to be correct.

```
precision = precision_score(y_true, y_pred)
print("Precision:", precision)
```

11.4 Recall

Recall or Sensitivity measures how many of the actual positive cases were correctly identified by the model. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

In scenarios where catching all positive cases is important (like disease detection), recall is a key metric.

```
recall = recall_score(y_true, y_pred)
print("Recall:", recall)
```

11.5 F1 Score

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number. A high F1 score means the model performs well on both metrics. Its range is [0,1].

Lower recall and higher precision gives us great accuracy but then it misses a large number of instances. More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
f1 = f1_score(y_true, y_pred)
print("F1 Score:", f1)
```

11.6 Area Under Curve (AUC) and ROC Curve

It is useful for binary classification tasks. The AUC value represents the probability that the model will rank a randomly chosen positive example higher than a randomly chosen negative example. AUC ranges from 0 to 1 with higher values showing better model performance.

ROC Curve is a graphical representation of the True Positive Rate (TPR) vs the False Positive Rate (FPR) at different classification thresholds. The curve helps us visualize the trade-offs between sensitivity (TPR) and specificity (1 - FPR) across various thresholds. Area Under Curve (AUC) quantifies the overall ability of the model to distinguish between positive and negative classes.

$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

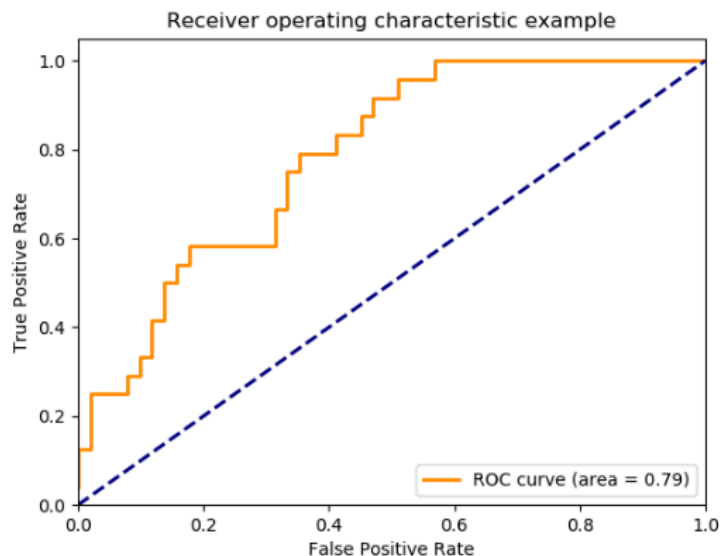
$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

- AUC = 1: Perfect model (always correctly classifies positives and negatives).
- AUC = 0.5: Model performs no better than random guessing.
- AUC < 0.5: Model performs worse than random guessing (showing that the model is inverted).

```
roc_auc = roc_auc_score(y_true, y_pred_proba)
print("ROC-AUC Score:", roc_auc)
```

ROC visualization:

```
fpr, tpr, thresholds = roc_curve(y_true, y_pred_proba)
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, color='blue', label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0,1], [0,1], color='red', linestyle='--')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```



11.7 Classification Report

A classification report is a summary table that shows how well your classification model performed, by giving key metrics Precision, Recall, F1-score, and Support for each class in your data.

```
report = classification_report(y_true, y_pred)
```

```
print("\nClassification Report:\n", report)
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.75         0.75         0.75         4
     1       0.75         0.75         0.75         4

 accuracy          0.75         8
 macro avg       0.75         0.75         0.75         8
 weighted avg    0.75         0.75         0.75         8
```

Reference:

<https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>